Operators

🎯 Session Objectives

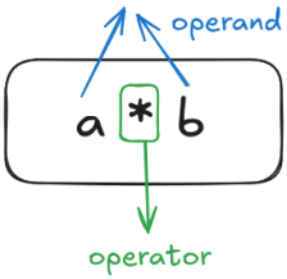
- 🔧 Understand what operators are and why they are used
- 🧮 Explore different types of operators in Python
- 🔢 Learn about operator precedence and order of execution
- ⚠️ Understand constraints in programming

What are Operators?

Operators are tools in programming used to perform actions like arithmetic, comparisons, assignments, logical evaluations, etc.

Types of Operators in Python:

1. Arithmetic Operators
2. Comparison Operators
3. Logical Operators
4. Bitwise Operators
5. Assignment Operators
6. Membership Operators
7. Identity Operators



Bitwise Operators Applications : Washing Machine, Refrigerators , AC

Bitwise Operators:

- 'AND' = '&' - 'Both bits with value 1 return 1 else 0'
- 'OR' = '|' - 'Any bit with value 1 return 1 else 0'
- 'XOR' = '^' - 'Alternative Bits returns 1 else same bit returns 0'
- 'NOT' = '~' - '2's complement to check the sign and evaluate'
- 'Left Shift' = '<<' - 'Shifts the bit to the left'
- 'Right Shift' = '>>' - 'Shifts the bit to the right'

bit1	bit2	result
1	1	1
1	0	0
0	1	0
0	0	0

AND & Bitwise

bit1	bit2	result
1	1	1
1	0	1
0	1	1
0	0	0

OR | Bitwise

bit1	bit2	result
1	1	0
1	0	1
0	1	1
0	0	0

XOR ^ Bitwise

10 decimal -> 1010 Binary

2	10
2	5 - 0
2	2 - 1
	1 - 0

2	10
2	5 - 0
2	2 - 1
2	1 - 0
	0 - 1

2^3	2^2	2^1	2^0
1	0	1	0

(1 * 2^3) + (0 * 2^2) + (1 * 2^1) + (0 * 2^0)
= (1*8) + (0*4) + (1*2) + (0*1)
= 8 + 0 + 2 + 0 = 10

& Bitwise

10 & 6

```
  1010
& 0110
------
  0010
```

2 in decimal

2	6
2	3 - 0
2	1 - 1
	0 - 1

bit1	bit2	result
1	1	1
1	0	0
0	1	0
0	0	0

& Bitwise

```python
# And Bitwise
x = 10
y = 6
print(x & y) # 2
```

```
2
```

OR | Bitwise

10 | 6

```
  1010
| 0110
------
  1110
```

14 in decimal

2	6
2	3 - 0
2	1 - 1
	0 - 1

bit1	bit2	result
1	1	1
1	0	1
0	1	1
0	0	0

OR | Bitwise

```python
# OR Bitwise
x = 10
y = 6
print(x | y) # 14
```

```
14
```

```
  010011
| 101001
--------
  111011
```

19 | 41

59 in decimal

```python
# OR Bitwise
x = 19
y = 41
print(x | y) # 59
```

```
59
```

XOR ^ Bitwise

10 ^ 6

```
  1010
^ 0110
------
  1100
```

12 in decimal

2	6
2	3 - 0
2	1 - 1
	0 - 1

bit1	bit2	result
1	1	0
1	0	1
0	1	1
0	0	0

XOR ^ Bitwise

```
# XOR Bitwise
x = 10
y = 6
print(x ^ y) # 12
```

```
12
```

NOT ~ Bitwise

~10 = -11

00001010

flip the bits

11110101

check the leftmost bit.

direction
1 : negative
0 : positive

1's Complement
+ 1

2's Complement

```
  1010
    +1
------
  1011
```

11 in decimal

$$\sim X = -(X+1)$$

```
# NOT Bitwise (~)
X = 10
print(~X) # -(X+1) # -11
```

```
-11
```

```
# NOT Bitwise (~)
X = 7
print(~X) # -(X+1) # -8
```

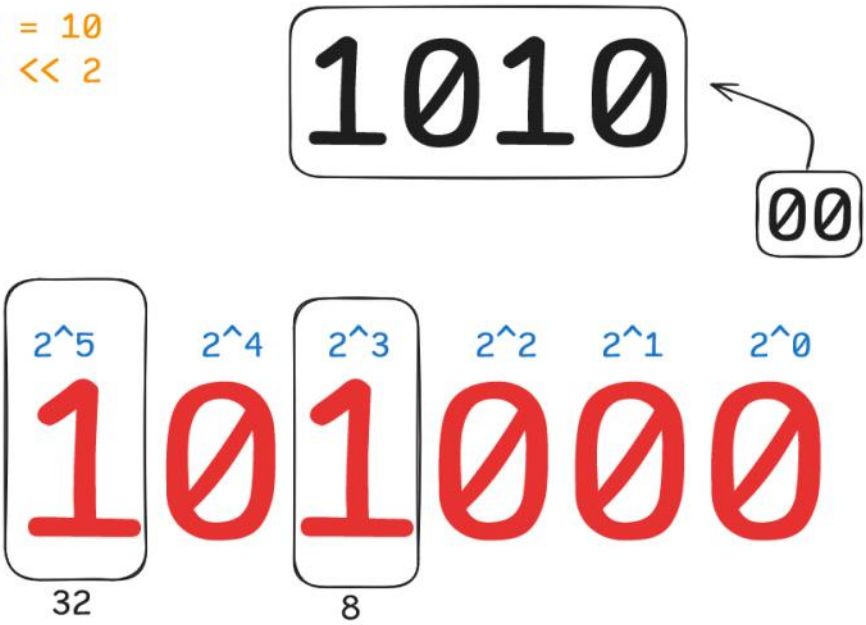
```
-8
```

```
# NOT Bitwise (~)
X = -17
print(~X) # -(X+1) # -(-17+1) = -(-16) = 16
```

```
16
```

Left Shift <<

X = 10
X << 2



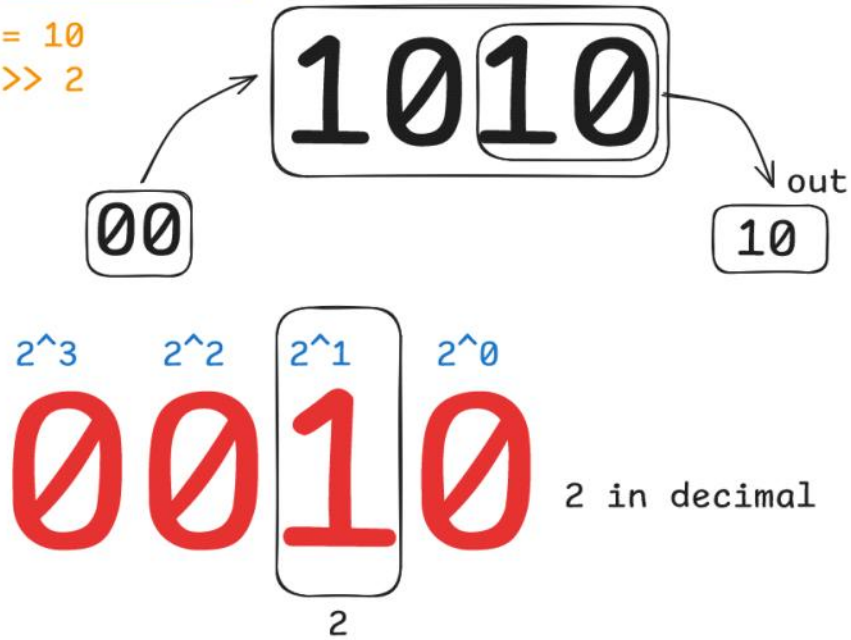
X = 10
X << 2
10 * 2^2 = 10 * 4 = 40

X << K [Left Shift]
X * 2^K

40 in decimal

Right Shift >>

X = 10
X >> 2



X = 10
X >> 2
10 // 2^2 = 10 // 4 = 2

X >> K [Right Shift]
X // 2^K

2 in decimal

```
# Right Shift (>>)
X = 10
print(X>>2) # 2
```

```
2
```

```
# Right Shift (>>)
X = 10
print(X>>3) # 10 // 2^3 = 10 // 8 = 1
```

```
1
```

Assignment Operators

Assignment Operators:

- '=' : (x = 5)
- '+=' : x+=5 => x = x + 5
- '-=' : x-=5 => x = x - 5
- '*=' : x *=5 => x = x * 5
- '/=' : x/=5 => x = x / 5
- '%=' : x%=5 => x = x % 5
- '//=' : x//=5 => x = x // 5
- '**=' : x ** =5 => x = x ** 5

```
x = 10
y = 11
z = 7
x += y
print(x)
x -= z
print(x)
x /= 2
print(x)
```

```
21
14
7.0
```

```
x *= x # x = x*x
print(x) # 49.0
```

```
49.0
```

```
x %= y # x = x % y # 49.0 % 11 - 5.0
print(x)
```

```
5.0
```

```
x //= z # x = x // z -> 5.0 // 7 -> 0.0
print(x)
```

```
0.0
```

```
y **= x # y = y ** x -> 11^0 = 1
print(y)
```

```
1.0
```

Membership Operators :

- It return the Boolean result.
- **'in'** operator -> `Return True if the value exist in a sequence`
- **'not in'** operator -> `Return True if the value doesn't exist in a sequence`

```
# Python is case-sensitive
print('hello' in 'hello World') # True
print('hello' in 'Hello World') # False
print('hello' not in 'Hello World') # True
```

```
True
False
True
```

```
print('I' in 'India') # True
print('I' not in 'America') # True
print('I' not in 'AmerIca') # False
```

```
True
True
False
```

```
print('mon' in ['Mon','Tue','Wed','Thurs','Fri']) # False
print('Sat' not in ['Mon','Tue','Wed','Thurs','Fri']) # True
print('Fri' not in ['Mon','Tue','Wed','Thurs','Fri']) # False
```

```
False
True
False
```

```
# Dictionary {Key : Values}
print('mon' in {'mon':'Mon', 'tues':'Tues'}) # True
print('Mon' in {'mon':'Mon', 'tues':'Tues'}) # False
```

```
True
False
```

JSON Structure

```
person1_details = {
'name' : ['Ankit','Shivam','Aman','Pardeep'],
'age' : [27,29,31,33],
'gender': ['Male','Male','Male','Male'],
'city': ['Mumbai','Delhi','Jaipur','Bangalore'],
'state': ['Maharashtra','Delhi',
          'Rajasthan' ,'Karnataka']
}
```

Name	Age	Gender	City	State
Ankit	27	Male	Mumbai	Maharastra
Shivam				
Aman				
Pradeep				

Identity Operators:

- **'is'** - `Returns True if both the variables refers to the same object (have same memory Address).`
- **'is not'** - `Returns True if both the variables refers to the different object (have different memory Address)`.

```python
# '==' Comparison Operators (data equality) 10 == 10 [Value Check]
# 'is' Compares the identity (object equality)

a = [1,2,3]
b = a # deep copy
c = [1,2,3] # independent data

print(a == b) # Data Evaluated
print(a is b) # Data + Address

print(a == c)
print(a is c)
```

```
True
True
True
False
```

```python
a = [1,2,3]
b = a # deep copy
c = ['a',2,3] # independent data

print(a == b) # True
print(a is b) # True

print(a == c) # False
print(a is c) # False
```

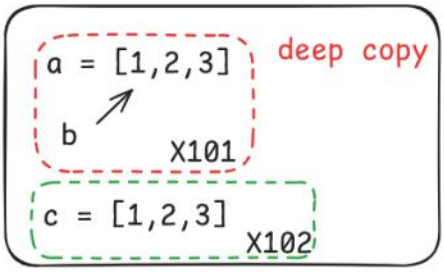
```
True
True
False
False
```

```python
a = [1,2,3]
b = a # deep copy
c = [1,2,3] # independent data

print(a == b) True
print(a is b) True

print(a == c) True
print(a is c) False
```

Memory Diagram



Deep Copy Vs Shallow Copy

Deep Copy

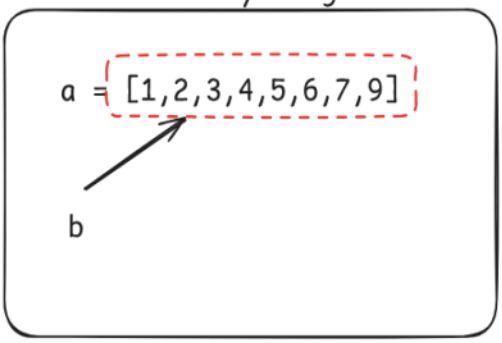
```
a = [1,2,3,4,5,6,7]
b = a # Deep Copy
b.append(9)
print(a == b) True
print(a is b) True
```

Shallow Copy

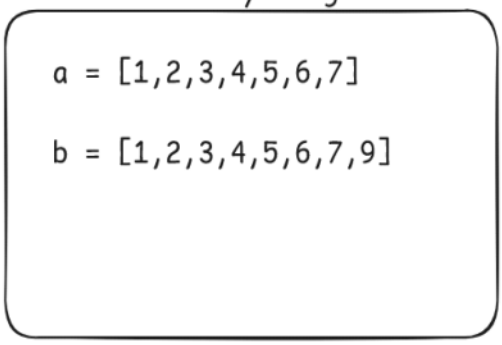
```
a = [1,2,3,4,5,6,7]
b = a.copy() # Shallow Copy
b.append(9)
print(a == b) False
print(a is b) False
```

independent table

Memory Diagram



Memory Diagram



b = a creates a variable b and points to same memory loacation of a
b = a.copy() creates a variable and creates a different memory location with same data as a.

```python
# Deep Copy
a = [1,2,3,4,5,6,7]
b = a # Deep Copy
b.append(9)
print(a == b)
print(a is b)
```

```
True
True
```

```python
a = [1,2,3,4,5,6,7]
b = a.copy() # Shallow Copy
b.append(9)
print(a == b)
print(a is b)
```

```
False
False
```